Informatique Théorique

CM11 : Codage des fichiers

Florian Bridoux

Polytech Nice Sophia

2024-2025

Table des matières

1. Taille et représentation des fichiers
2. Fichiers texte
3. Images
4. Fichiers audios
5. Manipulation des fichiers binaires en Python

Table des matières

1. Taille et représentation des fichiers
2. Fichiers texte
3. Images
4. Fichiers audios
5. Manipulation des fichiers binaires en Python

Représentation des données dans un ordinateur

Quelle que soit la nature de l'information traitée par un ordinateur (image, son, texte, vidéo), elle est toujours représentée sous la forme d'une suite de bits.

Exemple :

`0110 0001 0000 1010`

correspond à un fichier texte contentant uniquement la lettre *a*. **Remarque :**

- On dit parfois qu'un fichier n'est rien d'autre qu'un entier naturel (qu'on peut représenter en binaire).
- *Cependant :* `0000000010000001` n'est pas le même fichier que `10000001` donc ce serait plutôt un entier naturel + un nombre de bits.

La taille d'un fichier : Bits et Octets

Bits et octets :

- La taille d'un fichier est souvent exprimée en octets, pas en bits.

Unités standardisées (IEC, décembre 1998) :

- **1 kilooctet (Ko)** = 10^3 octets.
- **1 mégaoctet (Mo)** = 10^6 octets.
- **1 gigaoctet (Go)** = 10^9 octets.
- **1 téraoctet (To)** = 10^{12} octets.
- **1 pétaoctet (Po)** = 10^{15} octets.
- **1 exaoctet (Eo)** = 10^{18} octets.
- ...

La taille d'un fichier : Bits et Octets

Attention ! De nombreux logiciels (parfois même certains systèmes d'exploitation) utilisent toujours la notation antérieure à 1998 pour laquelle un kilooctet = 1024 octets. En réalités, on devrait parler de kibioctet...

- **1 kibioctet (Kio)** = 2^{10} = 1024 octets.
- **1 mébioctet (Mio)** = 2^{20} = 1 048 576 octets.
- **1 gibioctet (Gio)** = 2^{30} = 1 073 741 824 octets.
- **1 tébioctet (Tio)** = 2^{40} = 1 099 511 627 776 octets.
- ...

Éditeur hexadécimal

Même quand on fait une analyse "à la main" d'un fichier, on représente rarement le code binaire directement. En effet,

- La base binaire est très peu compacte ;
- Le lire serait très pénible ;
- Analyser moins d'un octet a rarement un intérêt.

D'un autre côté :

- Si on voulait le représenter un octet en un seul caractère (*i.e.* base 2^8), il nous faudrait 256 caractères différents...

L'hexadécimal est un compromis :

- On peut représenter un octet en seulement deux caractères hexadécimaux.
- Exemple : 11001101_2 = CD_{16} (`souvent noté 0xCD`).
- On trouve souvent des codes en hexadécimaux dans la documentation technique (par exemple de processeurs).

Little Endian vs Big Endian

Il existe deux grandes manières de stocker les nombres sur plusieurs octets.

- **Little Endian :**
  - **Définition :** L'octet de poids faible est stocké en premier (adresse la plus basse).
  - **Exemple (valeur : 0x12345678) :**
    - Stockage en mémoire : 78 56 34 12.
- **Big Endian :**
  - **Définition :** L'octet de poids fort est stocké en premier (adresse la plus basse).
  - **Exemple (valeur : 0x12345678) :**
    - Stockage en mémoire : 12 34 56 78.

L'Offset dans un Fichier

Qu'est-ce qu'un offset ?

- L'offset est la position relative d'une donnée dans un fichier ou une mémoire.
- Elle représente une distance en octets.
- Elle est souvent représenté en hexadécimale et peut aller de 0 (0x0) à la taille du fichier moins 1.

Table des matières

1. Taille et représentation des fichiers
2. Fichiers texte
3. Images
4. Fichiers audios
5. Manipulation des fichiers binaires en Python

Table ASCII

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	00	Null	32	20	Space	64	40	@	96	60	`
1	01	Start of heading	33	21	!	65	41	A	97	61	a
2	02	Start of text	34	22	"	66	42	B	98	62	b
3	03	End of text	35	23	#	67	43	C	99	63	c
4	04	End of transmit	36	24	$	68	44	D	100	64	d
5	05	Enquiry	37	25	%	69	45	E	101	65	e
6	06	Acknowledge	38	26	&	70	46	F	102	66	f
7	07	Audible bell	39	27	'	71	47	G	103	67	g
8	08	Backspace	40	28	(	72	48	H	104	68	h
9	09	Horizontal tab	41	29	)	73	49	I	105	69	i
10	0A	Line feed	42	2A	*	74	4A	J	106	6A	j
11	0B	Vertical tab	43	2B	+	75	4B	K	107	6B	k
12	0C	Form feed	44	2C	,	76	4C	L	108	6C	l
13	0D	Carriage return	45	2D	-	77	4D	M	109	6D	m
14	0E	Shift out	46	2E	.	78	4E	N	110	6E	n
15	0F	Shift in	47	2F	/	79	4F	O	111	6F	o
16	10	Data link escape	48	30	0	80	50	P	112	70	p
17	11	Device control 1	49	31	1	81	51	Q	113	71	q
18	12	Device control 2	50	32	2	82	52	R	114	72	r
19	13	Device control 3	51	33	3	83	53	S	115	73	s
20	14	Device control 4	52	34	4	84	54	T	116	74	t
21	15	Neg. acknowledge	53	35	5	85	55	U	117	75	u
22	16	Synchronous idle	54	36	6	86	56	V	118	76	v
23	17	End trans. block	55	37	7	87	57	W	119	77	w
24	18	Cancel	56	38	8	88	58	X	120	78	x
25	19	End of medium	57	39	9	89	59	Y	121	79	y
26	1A	Substitution	58	3A	:	90	5A	Z	122	7A	z
27	1B	Escape	59	3B	;	91	5B	[	123	7B	{
28	1C	File separator	60	3C	<	92	5C	\	124	7C	|
29	1D	Group separator	61	3D	=	93	5D	]	125	7D	}
30	1E	Record separator	62	3E	>	94	5E	^	126	7E	~
31	1F	Unit separator	63	3F	?	95	5F	_	127	7F	□

Table ASCII

Le code ASCII, permet de représenter des information textuelle. En ASCII, chaque caractère est codé sur 7 bits (c'est-à-dire 128 caractères possibles, de 0 à 127).

- Les codes 0 à 31 ne sont pas de vrais caractères. On les appelle caractères de contrôle car ils permettent de faire des actions telles que :
  - retour à la ligne (Carriage return) ;
  - bip sonore (Audible bell).
- Les codes 65 à 90 représentent les majuscules.
- Les codes 97 à 122 représentent les minuscules (il suffit de modifier le 6ème bit pour passer de majuscules à minuscules, c'est-à-dire ajouter 32 au code ASCII en base décimale).
- Le code ASCII a été mis au point pour la langue anglaise, il lui manque donc beaucoup de caractère nécessaire à la langue française comme les caractères accentués...

Qu’est-ce que UTF-8 ?

UTF-8 (Unicode Transformation Format - 8 bits) est un standard de codage des caractères qui permet de représenter l’ensemble des caractères du "répertoire universel de caractères codés".

- Les caractères de l’alphabet latin,
- Les symboles,
- Les écritures non latines (chinois, arabe, cyrillique, etc.),
- Et beaucoup d’autres caractères.

Caractéristiques principales :

- Utilise entre 1 et 4 octets par caractère.

Peut-être utilisée pour toutes les données textuelle :

- Codes sources de programmes.
- Pages web (HTML, CSS, JavaScript).
- Fichiers de configuration.
- Protocoles Internet (e-mails, API, etc.).

Pourquoi utiliser UTF-8 ?

Avantages :

- Format "universelle" supportant toutes les langues.
- Standard largement adopté sur le web et dans les bases de données.
- **Efficacité :**
  - Texte en anglais ou langues similaires est très compact (1 octet par caractère).

Désavantages :

- Pour certaines langues, peut-être plus volumineux que des ASCII étendue par exemple...

Fonctionnement de UTF-8

Codage des caractères :

- Les 128 caractères (ASCII) utilisent 1 octet.
- Les caractères supplémentaires utilisent entre 2 et 4 octets.

Structure d'un caractère :

- **1 octet :** `0xxxxxxx` (ASCII).
- **2 octets :** `110xxxxx 10xxxxxx`.
- **3 octets :** `1110xxxx 10xxxxxx 10xxxxxx`.
- **4 octets :** `11110xxx 10xxxxxx 10xxxxxx 10xxxxxx`.

Fonctionnement de UTF-8

Exemple :

- Caractère A (ASCII) : 1 octet (01000001).
- Caractère é (Unicode) : 2 octets (11000011 10101001).
- Caractère λ (Lettre grecque lambda) : 3 octets (11101100 10111010 10111011).
- Caractère 丈 : 4 octets (11110000 10100000 10000000 10101011).
- Caractère 😀 (emoji) : 4 octets 11110000 10011111 10011000 10000000.

Table des matières

1 Taille et représentation des fichiers

2 Fichiers texte

3 Images

4 Fichiers audios

5 Manipulation des fichiers binaires en Python

Types de fichiers images

Il y a différents types de fichiers images :

- Format non compressé, chaque pixel est représenté directement par des valeurs de couleur. Exemple **BMP** (pour Bitmap).
- Format compressé sans perte. (une image "normal" sera souvent plus légère mais une image "aléatoire" sera alors plus lourde). Exemple : **PNG** (Portable Network Graphics).
- Format compressé avec perte : les photos (par exemple) sont dégradées pour obtenir un fichier plus léger, mais de manière à être le moins perturbant possible pour l'il humain. Exemple : **JPEG** (Joint Photographic Experts Group).

Les entêtes des formats d'image

Les formats d'image tels que BMP, PNG et JPEG commencent tous par une entête qui contient des informations essentielles sur l'image. Ces entêtes permettent de comprendre la structure du fichier et de l'interpréter correctement (dimension de l'image, encodage, type de compression, métadonnées...).
Ils commencent par une signature qui permet d'indiquer le format du fichier image.
En hexadécimal :

- BMP : 42 4D (BM en UTF-8)
- PNG : 89 50 4E 47 0D 0A 1A 0A (50 4E 47 est PNG en UTF-8)
- JPEG : 0xFF 0xD8 0xFF

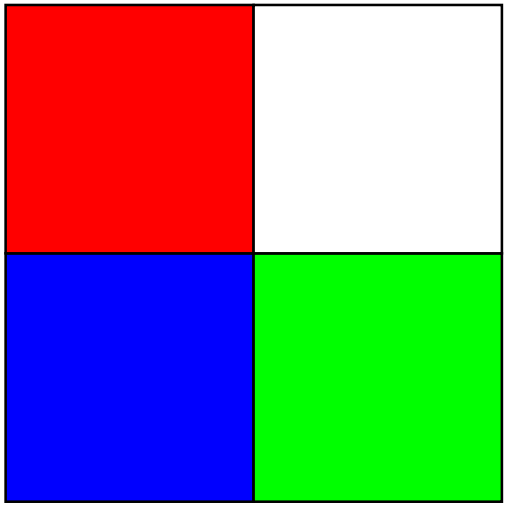
Format BMP

- Encodage des pixels en **BGRA** : Le format d'encodage des pixels que nous allons voir en TD.
- Un format de couleur où chaque pixel est représenté par 4 octets.
- Chaque octet correspond à un composant de couleur et de transparence :
  - **B** (Blue) : Composant bleu de la couleur.
  - **G** (Green) : Composant vert de la couleur.
  - **R** (Red) : Composant rouge de la couleur.
  - **A** (Alpha) : Composant de transparence (0 = transparent, 255 = opaque).
- Chaque composant est codé sur 1 octet (8 bits), soit une valeur entre 0 et 255.
- Exemple de pixel codé en BGRA :
  - 63 73 33 FF : où **bleu :** $(63)_{16} = 99$ (sur 255) , **vert :** $(73)_{16} = 115$, **rouge** :$(33)_{16} = 51$, **FF** = 255 (opaque).

Format BMP

Disposition des pixels :

- Les pixels sont disposés de **gauche à droite** et de **bas en haut**.



Code de la parties (pixel) de l'image :

FF0000FF 00FF00FF 0000FFFF FFFFFFFF

Table des matières

1. Taille et représentation des fichiers
2. Fichiers texte
3. Images
4. Fichiers audios
5. Manipulation des fichiers binaires en Python

Format WAV

- WAV, contraction de *WAVEform Audio Format*, est un standard de Microsoft et IBM pour stocker l'audio numérique.
- Le format WAV est un conteneur capable de recevoir des formats variés comme le MP3, le WMA, l'ATRAC, ou le PCM.
- Le format PCM est le plus courant :
  - Données brutes, ni modifiées ni compressées.
  - En-tête de 44 octets contenant des informations comme :
    - Format audio.
    - Fréquence d'échantillonnage.
    - Nombre de canaux.

Structure d'un fichier WAV

- Un fichier WAV est divisé en trois blocs principaux :
  - **Bloc de déclaration** : identifie le fichier comme un WAV.
  - **Bloc de description du format** : spécifie les détails audio.
  - **Bloc des données** : contient les données audio brutes.
- L'en-tête dispose de 44 octets.

Détails de la structure

Nom	Octets	Type	Endian	Description
Bloc de déclaration				
Chunk ID	4	4 char	Big	`RIFF` (ASCII, 0x52494646)
Chunk Size	4	int 32	Little	Taille du fichier - 8 octets
Format	4	4 char	Big	`WAVE` (ASCII, 0x57415645)
Bloc de description du format audio				
SubChunk 1 ID	4	4 char	Big	`fmt` (ASCII, 0x666d7420)
SubChunk 1 Size	4	int 32	Little	Taille = 16 pour PCM
Audio Format	2	int 16	Little	Type de compression (1 = PCM)
Num Channels	2	int 16	Little	Nombre de canaux (1 = Mono, 2 = Stéréo)
Sample Rate	4	int 32	Little	Fréquence d'échantillonnage
Byte Rate	4	int 32	Little	`Fréquence * Bits/Canal/8 * NbCanaux`
Block Align	2	int 16	Little	`NbCanaux * NbOctets/Canal`
Bits Per Channel	2	int 16	Little	Nombre de bits par canal
Bloc des données				
SubChunk 2 ID	4	4 char	Big	`data` (ASCII, 0x64617461)
SubChunk 2 Size	4	int 32	Little	`NbCanaux * NbSamples * Bits/Canal/8`
Data	n - 44	...	Little	Données audio brutes

Table des matières

1. Taille et représentation des fichiers
2. Fichiers texte
3. Images
4. Fichiers audios
5. Manipulation des fichiers binaires en Python

Ouverture et fermeture des fichiers binaires

Pour travailler avec des fichiers en Python, nous utilisons :

- la fonction `open()` pour ouvrir un fichier ;
- la méthode `close()` pour le fermer.

Exemple classique d'ouverture et de fermeture :

```
1 f = open('exemple', 'r')
2 bin = f.read()
3 f.close()
```

Utilisation du mot-clé `with`

L'utilisation de `with` garantit que le fichier est correctement fermé, même en cas d'exception.

Exemple :

```
1 with open('exemple', 'r') as f:
2     bin = f.read()
3 # Le fichier est automatiquement fermé ici
```

Les modes d'ouverture des fichiers en Python

Lorsqu'on travaille avec des fichiers en Python, différents modes peuvent être utilisés pour les ouvrir :

- `'r'` : Lecture **(Read)** Ouvre le fichier en mode lecture.
- `'w'` : Écriture **(Write)** Ouvre le fichier en mode écriture (écrase le contenu existant).
- `'a'` : Ajout **(Append)** Ouvre le fichier en mode ajout (écrit à la fin du fichier).

Exemples :

```
1 with open('exemple.txt', 'r') as f:
2     contenu = f.read()
3 with open('exemple.txt', 'w') as f:
4     f.write('Nouveau contenu')
5 with open('exemple.txt', 'a') as f:
6     f.write('\nContenu ajoute')
```

Différence entre mode texte et mode binaire

En mode texte ('r', 'w', 'a'), le contenu d'un fichier est lu écrit caractère par caractère.

Exemple :

```
1 # Si le fichier contient : "Ça a été."
2 with open('montexte.txt', 'r') as f:
3     for t in f.read():
4         print(t)
```

Affiche (un par un) : Ç,a, ,a,é,t,é. Or, le contenu réel du fichier

en code hexadécimal est : `C3 87 61 20 61 20 C3 A9 74 C3 A9 2E 0A`.
Le caractère Ç est représenté sur **2 octets** (`C3 87`).

Lecture en mode binaire

Pour lire un fichier **octet par octet**, il faut l'ouvrir en mode binaire (`'rb'`).

Exemple :

```
1 with open('montexte.txt', 'rb') as f:
2     for t in f.read():
3         print(hex(t))
4 # Affichage :
5 # 0xc3\n0x87\n0x61\n0x20\n0x61\n0x20\n0xc3\n0xa9\n0x74\
    n0xc3\n0xa9\n0x2e\n0xa\n
```

En mode binaire, chaque octet est lu individuellement, sans conversion en caractères. (f.read() renvoie un type bytes).

Les types bytes et bytearray en Python

En Python, bytes et bytearray permettent de représenter des séquences de valeurs d'octets (0-255).

Type bytes

- **Immuable** : Une instance de bytes ne peut pas être modifiée après sa création.
- **Création** :

```
1 b = bytes([65, 66, 67])  # Liste d'entiers
2 print(b)  # b'ABC'
3
4 b = b"ABC"  # Littéral d'octets
```

- **Exemple d'immuabilité** :

```
1 b = b"hello"
2 b[0] = 72  # TypeError: les objets bytes sont
      immuables
```

Type bytearray

Type bytearray

- **Mutable** : Les instances de bytearray peuvent être modifiées après leur création.
- **Création** :

```
1 ba = bytearray([65, 66, 67])  # Liste d'entiers
2 print(ba)  # bytearray(b'ABC')
3 ba = bytearray(b"ABC")  # Depuis un objet bytes
```

- **Exemple de modification** :

```
1 ba = bytearray(b"hello")
2 ba[0] = 72  # Remplace 'h' par 'H'
3 print(ba)  # bytearray(b'Hello')
```

Les méthodes read() et read(n) en Python

Attention : Appeler read() deux fois de suite ne lit pas deux fois le fichier depuis le début. La position du curseur dans le fichier avance après chaque lecture.

- read() lit tout le contenu restant du fichier depuis la position actuelle du curseur et déplace le curseur à la fin.
- Si le fichier est volumineux, cela peut consommer beaucoup de mémoire...

```
1 with open('file.txt', 'r') as file:
2     content = file.read()  # Lit tout le fichier
3     print(content)
```

Les méthodes read() et read(n) en Python

- read(n) lit jusqu'à n octets ou caractères (selon le mode d'ouverture) depuis la position actuelle du curseur.
- Si n est plus grand que le contenu restant, il lit jusqu'à la fin.

```
1 with open('file.txt', 'r') as file:
2     part = file.read(5)  # Lit les 5 premiers caractères
3     print(part)
```

Les méthodes seek() et tell() en Python

Fonctions utiles :

- seek(of) : met le curseur à la position de l'offset of.
- tell() : Obtenir la position actuelle du curseur

Exemple :

```
1 print(file.tell())  # Affiche '0'
2 file.read() # Affiche le contenu du fichier
3 print(file.tell())  # Affiche la taille du fichier -1
4 file.seek(10)       # Déplace le curseur 10 octets
      depuis le début
5 print(file.tell())  # Affiche '10'
```